

Directory

1. Introduction	1
2. Methodology	1
2.1 MinMaxScaler normalization processing	2
2.2 Principal component analysis (PCA)	2
2.3 Grid Search Cross-Validation (GridSearchCV)	2
2.4 K-NN (K-nearest neighbor)	3
2.5 Support Vector Machine (SVM)	3
2.6 Decision tree	3
2.7 Bagging ensemble	3
3. Result and Discussion	4
3.1 Search hyperparameter	4
3.2 K-NN Result	4
3.3 SVM Result	5
3.4 Decision Tree Result	5
3.5 Bagging with SVM	6
3.5 Discussion	6
4. Conclusion	8
Reference	9
Appendix	10

1. Introduction

This report begins with a description of the data set. Describes the selected algorithm and the rationale for this algorithm. Finally, the key findings and results are summarized.

The dataset contains 30,000 training samples and 2,000 test data. 5,000 final data to be predicted, each sample represented by 784 features (i.e., a 28x28 pixel grayscale image). Ten different categories need to be identified. No validation set is provided, but 10% of the data from the training set can be randomly divided for validation.

Four machine learning algorithms are selected for this report: K-nearest neighbor algorithm (K-NN), support vector machine (SVM), decision tree and Bagging ensemble. These three basic models have the ability to handle high dimensional data and are suitable for complex classification. Naive Bayes have the limitation of independent assumptions, which are not suitable for this task. Logistic regression model can not deal with some complex boundary problems and is not suitable for complex nonlinear classification problems. Therefore, those two basic models, Bayesian and logistic regression, were not chosen.

The report analysis focuses on evaluating the performance of each model in gray-scale image classification, using performance indicators such as predict accuracy, accuracy and recall rate to compare. Through this comparison, the most suitable model is determined.

The key findings are summarized below:

1. K-NN: As a simple instance-based method, it performs well but is not computationally efficient because the distance of all training samples needs to be calculated.
2. SVM: Excellent at dealing with complex boundaries, but computatively expensive, especially on large data sets.
3. Decision tree: has intuitive decision boundaries, but is easy to overfit, and proper tuning can significantly improve accuracy.
4. Bagging integration method: The SVM performance is improved, but the calculation time is too high and the performance improvement is limited.

The following sections will analyze the performance of these methods in detail and compare the results of the training set , validation set and the test set.

2. Methodology

First, we train individual models of K-NN, SVM, and Decision tree. Then GridSearchCV is used to find the optimal hyperparameters on the basis of a single model. Finally, Bagging ensemble method is used to combine

the single model with the best performance to obtain higher accuracy. The data processing methods are used as follows: 1. MinMaxScaler normalization processing 2. Principal component analysis (PCA).

2.1 MinMaxScaler normalization processing

MinMaxScaler is a normalization technique by scaling features to a given range, usually [0, 1]. It ensures that all features are equally important in model training, avoiding the extreme value to affect the final result. Observing the data set, we can see that the feature value of the position without pixels is 0, and the feature value of the point with pixels is very large, so normalization processing is needed to avoid the impact of extreme values on the results.

$$X_{\text{new}} = \frac{X - \min(x)}{\max(x) - \min(x)}$$

Normalization helps to avoid the effect of extreme values of features on the model. Especially in distance-based algorithms such as K-NN and SVM, normalization can improve the convergence speed and classification accuracy. According to Pandey & Jain (2017), it can be seen that reasonable use of MinMAXScale method under KNN can significantly improve the accuracy of KNN. Therefore, the normalized processing method is chosen to process the data.

2.2 Principal component analysis (PCA)

Map the data so that each column represents a dimension of the data. Calculate the mean of each dimension. Computes the covariance matrix of the input data matrix, which represents the linear correlation between the dimensions. The eigenvalues and eigenvectors are calculated to obtain the principal components. By retaining the eigenvector corresponding to the first K maximum eigenvalues, dimension reduction is realized to form the data set after dimensionality reduction (George, 2012).

$$\text{Final Data} = \text{RowFeatureVector} \times \text{RowDataAdjust}$$

PCA reduces features on high-dimensional data sets, improves computational efficiency, and reduces the risk of overfitting. However, for image datasets, dimensionality reduction may not significantly improve accuracy, as pixel-level features can be very important.

2.3 Grid Search Cross-Validation (GridSearchCV)

GridSearchCV is a hyperparameter tuning technique that finds the best combination of hyperparameters by traversing predefined parameter combinations. Use cross-validation to evaluate the performance of each set of parameters, ensuring robust results and avoiding overfitting. k stands for K-fold, and i stands for accuracy each time. The average accuracy is found to represent the accuracy of the model under this parameter.

$$\text{MeanAccuracy} = \frac{1}{k} \sum i$$

GridSearchCV ensures that the selected combination of hyperparameters provides the best model performance. It helps avoid overfitting or underfitting the model by traversing the search for parameter values.

2.4 K-NN (K-nearest neighbor)

K-nearest Neighbor (KNN) technology is widely used in data mining and machine learning due to its simplicity and superior performance. Classification is used for labeling after training sample data. Many classification methods have been proposed by researchers over the past few decades, but KNN remains one of the most popular methods for classifying datasets (Pandey & Jain, 2017). According to (Pandey & Jain, 2017), the calculation method of KNN is to calculate the distance between two points with the following formula:

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_2 - y_1)^2}$$

K-nn is a parameterless instance-based learning algorithm that classifies according to the majority classes of the K nearest neighbors in a feature space. Distance metrics (e.g., Euclidean distance, Manhattan distance) are used to determine the nearest point.

K-NN is simple and suitable for small data sets, especially when the decision boundary is complex and non-linear. It does not depend on data distribution assumption and can deal with multiple classification problems efficiently. However, for large data sets, it is computationally expensive.

2.5 Support Vector Machine (SVM)

SVM is a supervised learning algorithm that separates classes by maximizing the distance between the decision boundary and the nearest data point (support vector) and finding the optimal hyperplane. SVM can handle both linear and nonlinear classification by using kernel functions.

SVM is very powerful for high-dimensional data and complex decision boundaries. It enhances generalization by maximizing spacing. However, SVM is computationally expensive for large data sets.

2.6 Decision tree

A decision tree is a parameterless algorithm that recursively divides a dataset into subsets. It creates a tree structure where each node represents a feature-based division and leaf nodes represent category labels.

Decision trees are easy to explain. It is suitable for classification and regression tasks and performs well on both categorical and numerical data. However, decision trees are easy to overfitting, especially if the tree depth is large. This can be mitigated through pruning or integration methods.

2.7 Bagging ensemble

Bagging is an integrated approach that improves model accuracy and stability by training multiple base models on different training data. Final majority vote (classification) results.

Bagging reduces the variance of unstable models like decision trees by voting on the results of multiple model predictions. It can improve model generalization and reduce overfitting, especially in unstable models such as decision trees.

3. Result and Discussion

3.1 Search hyperparameter

Use GridSearchCV for hyperparameter search. The hyperparameter grid for each model is as follows:

```
1. K-NN : param_grid = {'n_neighbors': [2,3,5,7,9,11,13],
                        'p':[1,2]}
```

n_neighbors and p, which are the number and distance of neighbors.

```
2. SVM : param_grid = { 'C': [0.01, 0.1, 1, 10, 100],
                        'degree': [2, 3, 4],
                        'gamma': ['scale', 'auto', 0.1, 1]}
```

C (penalty parameter), degree (degree of the polynomial kernel), and gamma (kernel coefficient).

```
3. Decision Tree:  param_grid = {
                        'max_depth': [5,10,15,20],
                        'min_samples_split': [5,10,15,20],
                        'min_samples_leaf': [1,3,5,7,9],
                        'criterion': ['gini', 'entropy']}
```

max_depth (maximum tree depth), min_samples_split (minimum number of samples required for node splitting), and min_samples_leaf (minimum number of samples for leaf nodes).

3.2 K-NN Result

Best hyperparameters: n_neighbors=5, p=1

Validation set accuracy: 0.855

Test set accuracy: 0.8585

accuracy, recall and f1-score:

	precision	recall	f1-score	support
0	0.79	0.89	0.84	191
1	0.98	0.96	0.97	199
2	0.71	0.79	0.75	191
3	0.87	0.86	0.87	174
4	0.78	0.81	0.80	203
5	0.98	0.85	0.91	210
6	0.72	0.59	0.65	201
7	0.85	0.97	0.91	212
8	0.98	0.93	0.95	216
9	0.92	0.94	0.93	203
accuracy			0.86	2000
macro avg	0.86	0.86	0.86	2000
weighted avg	0.86	0.86	0.86	2000

3.3 SVM Result

In the SVM model, PCA was used to reduce the dimensionality of the data. The goal is to reduce redundant information in 784-dimensional features and speed up computation. However, the results show that the effect of using normalization treatment is better than that of PCA treatment. This may be because grayscale image features are important, but PCA dimensionality reduction will lose useful information.

The accuracy was 0.870 with common parameters (kernel= 'poly', degree=2,)

Therefore, the best parameters found using the normalization process are:

Best hyperparameters: C=10, degree=2, gamma='scale',kernel='poly'

Validation set accuracy: 0.8837

Test set accuracy: 0.8930

accuracy,recall and f1-score:

	precision	recall	f1-score	support
0	0.81	0.87	0.84	191
1	0.97	0.99	0.98	199
2	0.82	0.85	0.84	191
3	0.90	0.90	0.90	174
4	0.86	0.86	0.86	203
5	0.95	0.93	0.94	210
6	0.77	0.69	0.73	201
7	0.92	0.93	0.92	212
8	0.99	0.95	0.97	216
9	0.92	0.95	0.93	203
accuracy			0.89	2000
macro avg	0.89	0.89	0.89	2000
weighted avg	0.89	0.89	0.89	2000

3.4 Decision Tree Result

Best hyperparameters: max_depth=10, min_samples_split=10, min_samples_leaf=3

Test set accuracy: 0.81

accuracy, recall and f1-score:

	precision	recall	f1-score	support
0	0.75	0.77	0.76	295
1	0.96	0.92	0.94	300
2	0.70	0.74	0.72	298
3	0.81	0.83	0.82	312
4	0.69	0.72	0.70	292
5	0.91	0.85	0.88	299
6	0.59	0.55	0.57	294
7	0.84	0.93	0.88	287
8	0.94	0.91	0.92	324
9	0.94	0.90	0.92	299
accuracy			0.81	3000
macro avg	0.81	0.81	0.81	3000
weighted avg	0.81	0.81	0.81	3000

3.5 Bagging with SVM

Validation set accuracy: 0.8843

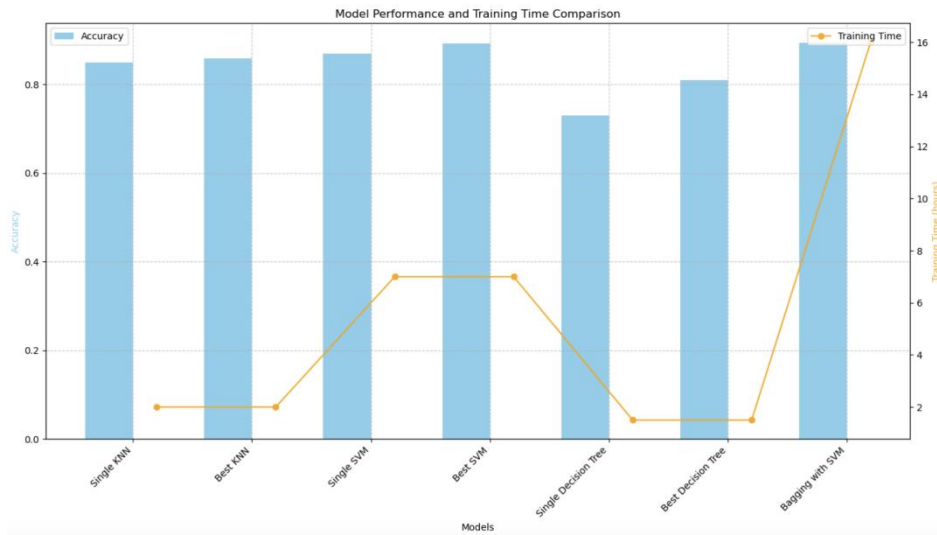
Test set accuracy: 0.8935

accuracy, recall and f1-score:

	precision	recall	f1-score	support
0	0.84	0.88	0.86	191
1	0.97	0.99	0.98	199
2	0.81	0.84	0.83	191
3	0.89	0.91	0.90	174
4	0.88	0.85	0.86	203
5	0.94	0.92	0.93	210
6	0.78	0.71	0.74	201
7	0.92	0.93	0.93	212
8	0.98	0.94	0.96	216
9	0.91	0.94	0.93	203
accuracy			0.89	2000
macro avg	0.89	0.89	0.89	2000
weighted avg	0.89	0.89	0.89	2000

3.6 Discussion

SVM achieves the highest accuracy (0.8930) with a training time of 7 hours, offering a good balance between performance and time. Using Bagging with SVM slightly improves accuracy (0.8935), but the training time significantly increases to 16 hours, making it a less ideal result for the model. The trade-off between input and output is disproportionate. We can see how each model performs in the images below.



Decision Tree may not perform as well as the SVM models, but its return on investment in terms of performance is more favorable. KNN models offer good accuracy with moderate training times, making them a middle-ground option. In the table below, we can see the time complexity of each model:

Model	Training time complexity	Predict time complexity	Explain Parameter
K-NN	$O(1)$	$O(T * N * d)$	T: Number of test set samples N: Number of training set samples d: feature dimension
SVM(ploy)	$O(N^2 * d)$	$O(S * d)$	N: Number of training set samples S: Number of support vectors d: feature dimension
Decision Tree	$O(N * d * \log N)$	$O(\log N)$	N: Number of training set samples d: feature dimension
Bagging	$O(M * N^2 * d)$	$O(M * S * d)$	M: Number of base models in Bagging N: Number of training set samples d: feature dimension S: Number of support vectors

If training time is the top priority, KNN or Decision Tree would be better choices. However, if accuracy is the main focus, Best SVM or Bagging with SVM should be prioritized. The final decision depends on the specific requirements of the situation. This provides a balanced approach, depending on whether you prioritize training time or accuracy for your project.

Decision Trees didn't performed well in this experiment, the reasons we've discussed shows below:

- Each sample in data contains 784 features. The large feature space is a challenge for decision trees since the high dimensionality may cause overfitting. Decision trees make decisions by splitting data on feature values, but some of the feature values might not have helpful information for the decision tree to identify

the feature and split them, and it would create a 'noise' node which is useless for the prediction. The 'noise' node would also be used by decision trees to make splits which cause overfitting.

- Decision trees treat each feature as an independent object, and split them based on each 'individual's' feature value. However, the neighboring features(pixel) may relate to each other to form a pattern of the image, and the decision tree ignores the relationship which makes the recognition hard.
- In the grayscale image dataset, each feature may have the important information which would impact the final image recognition, but the decision tree may ignore the necessary component of the image. In order to avoid overfitting, limits the max depth of the decision tree may make it worse since the model is not able to capture enough information to recognize the image.

For further improvement for decision trees. Ensemble methods could be applied such as Random Forest or Gradient Boosting to combine multiple decision trees to reduce overfitting and increase the accuracy; these models could better deal with the 'noise' node in grayscale image dataset.

SVM has the best performance in this experiment, reasons discussed below:

- Compared to the decision tree, SVM separates classes by maximizing the distance between the decision boundary and the nearest data point, and it focuses the most challenging samples at boundaries to reduce the impact of 'noise' data.
- SVM focuses on the data points that are closest to the decision boundary to find the optimal hyperplane, it could help SVM to avoid the risk of overfitting.
- The high dimension doesn't impact the performance of SVM, if the margin between classes is clear, SVM could find separating boundaries in high dimensional space through the kernel functions. Even if the data is non-linearly separable.

4. Conclusion

This report analyzes the gray-scale image classification task and compares the performance of four machine learning algorithms, K-NN, SVM, decision tree and bagging, under different preprocessing and hyperparameter optimization strategies

Key findings include: SVM performs best with a test set accuracy of 0.8935. It is excellent at dealing with complex nonlinear boundaries but it has long training and prediction times; K-NN has low accuracy and long training time, which limits its large-data application. When bagging ensemble combined with SVM, the training time become long, however, the improvement of accuracy is not obvious.

The limitations of the report include long training and predict time of SVM, and high computational complexity of K-NN. Decision tree has low accuracy, and limited accuracy of bagging improvement. Future directions of work include the introduction of more complex ensemble models (such as AdaBoost), optimization of model training, and using Gpus for accelerated training. Expand the dataset size and dimension, and try more dimension reduction techniques to improve model performance and application value.

Reference

1. George, A. (2012). Anomaly Detection based on Machine Learning Dimensionality Reduction using PCA and Classification using SVM. *International Journal of Computer Applications*, 47(21), 5–8.
<https://doi.org/10.5120/7470-0475>
- 2, Pandey, A., & Jain, A. (2017). Comparative Analysis of KNN Algorithm using Various Normalization Techniques. *International Journal of Computer Network and Information Security*, 9(11), 36–42.
<https://doi.org/10.5815/ijcnis.2017.11.04>

Appendix

This appendix provides detailed instructions for setting up the environment to run the code, including installation steps, required external libraries, and hardware configurations.

1. hardware configurations

- Operating System: macOS
- Python Version: 3.8 or higher
- Disk Space: At least 8 GB of free space for datasets and libraries
- Processor: Apple M2

2. External Libraries and Packages

The following external packages and libraries are required for run the code. You can install them using the pip:

Required Libraries:

- NumPy: For numerical computations
- Pandas: For data manipulation
- Scikit-learn: For machine learning models (K-NN, SVM, Decision Tree, Random Forest, etc.)
- Matplotlib: For data visualization
- Jupyter: Optional, for running notebooks

To install command:

```
# Install NumPy, Pandas, Matplotlib
```

```
pip install numpy pandas matplotlib
```

```
# Install Scikit-learn for machine learning models
```

```
pip install scikit-learn
```

```
# Install Jupyter (optional, if you plan to use Jupyter Notebooks)
```

```
pip install jupyter
```

3. Running the Code

To run the code, follow these steps:

1. download the project files
2. Open the Anaconda
3. Install the required packages or library as listed above.
4. Run the Jupyter Notebook